

Neural Connectomics in Silicon: High-Fidelity Simulation, Embodied Control, and Whole-Brain Modeling of *Drosophila melanogaster* Connectomic Foundations and Biomechanical Digital Twins

The computational reconstruction of the fruit fly nervous system represents a watershed transition in systems neuroscience, shifting the field from localized microcircuit models toward embodied, whole-brain simulations¹. For decades, computational neuroscientists relied on organisms with minimal neural architectures, such as the 302-neuron nervous system of *Caenorhabditis elegans* or isolated invertebrate ganglia, to explore the relationship between wiring and dynamics³. The emergence of high-throughput serial-section transmission electron microscopy, automated computer vision pipelines, and distributed proofreading platforms has enabled the complete mapping of synaptic connectomes in *Drosophila melanogaster*¹. Modern *in silico* fly neuroscience is anchored by two foundational datasets that capture different biological scopes:

The Full Adult Flying Brain (FAFB) dataset, curated through the FlyWire consortium, reconstructs the central brain of an adult female fly, cataloging approximately 139,255 neurons and over 50 million synaptic connections². FlyWire provides an exhaustive wiring diagram of sensory integration hubs, the visual optic lobes, the olfactory antennal lobes, the associative mushroom bodies, and the central complex navigational center³.

Complementing this, the MaleCNS dataset, released collaboratively by Google Research and Howard Hughes Medical Institute's (HHMI) Janelia Research Campus, reconstructs the complete central nervous system of an adult male fly⁵. MaleCNS encompasses 166,700 annotated neurons, 11,710 distinct cell types, and more than 25.5 million directed synaptic connections representing roughly 125 million individual contacts⁵. Crucially, MaleCNS bridges the central brain and the Ventral Nerve Cord (VNC) through the cervical connective⁴. Because the VNC functions as the insect analog of the vertebrate spinal cord, containing the motor neurons and premotor interneurons that actuate the appendages, MaleCNS enables true end-to-end modeling from sensory detection to physical execution⁴.

To ground these neuronal graphs in physical reality, computational frameworks interface the connectome with biomechanically accurate virtual bodies⁹. NeuroMechFly v2, developed by the Ramdya Laboratory at EPFL and distributed through the FlyGym framework, constitutes a data-driven digital twin derived from micro-computed tomography (μCT) scans of an adult female fly⁹. NeuroMechFly incorporates articulated skeletal segments with physiologically

bounded leg joint kinematics, adhesive pads on the pretarsi to capture surface traction, compound eyes resolved into an ommatidial lattice, and antennal olfactory receptors⁸. In parallel, the Flybody framework, developed jointly by Janelia, Google DeepMind, and the Turaga Lab, provides an anatomically detailed fly model optimized for the MuJoCo physics engine¹¹. Flybody integrates actuated wings, abdominal segments, and legs, making it particularly suitable for reinforcement learning workflows across terrestrial walking and aerial flight regimes¹¹.

Simulation Substrate / Connectome Dataset	Anatomical Coverage	Synaptic Contact Count	Primary Physics Engine	Key Sensory-Motor Modalities
FlyWire (FAFB v783) [cite: 2, 14]	Adult female central brain (139,255 neurons) ²	~ synapses ²	Custom graph simulators; Brian2; PyTorch ¹³	Ommatidial vision; antennal olfaction; taste; mechanosensation ²
MaleCNS (v1.0) [cite: 5, 6]	Adult male whole CNS: brain + VNC (166,700 neurons) ⁵	~ contacts across 25.58M directed pairs ⁶	Native C++ kernels; CUDA; WebGPU; Apple Metal ⁷	End-to-end sensorimotor loops; descending motor control ⁷
NeuroMechFly v2 (FlyGym) [cite: 9, 12]	Biomechanical digital twin from μ CT scans ⁹	Interfaces with sub-graphs or full connectomic matrices ⁸	MuJoCo; NVIDIA Warp / MJWarp ⁸	Hexagonal ommatidia vision; plume olfaction; leg adhesion ⁸
Flybody [cite: 11, 13]	Multi-segment cuticular body mesh ¹¹	Connects to RL policies or connectome-constrained networks ¹⁰	DeepMind MuJoCo ¹¹	Actuated wings; abdominal pitch; multi-joint tripod locomotion ¹¹

Biophysical Dynamics and Functional Network

Modeling

A structural connectome records anatomical adjacency and synapse numbers, but it does not inherently reveal membrane time constants, reversal potentials, or neurotransmitter dynamics¹⁸. Bridging the gap between static anatomy and active neurophysiology requires formal mathematical models that assign polarities and temporal equations to every node¹⁹.

Neurotransmitter Inference and Signed Adjacency Formulations

Electron microscopy volumes do not directly reveal the chemical identity of individual synapses¹⁰. To overcome this limitation, deep convolutional networks trained on ultrastructural features—such as synaptic vesicle morphology, active zone geometry, and mitochondrial density—predict neurotransmitter classifications across every reconstructed pre-synaptic site¹⁰. In whole-brain implementations established by Shiu et al., these biochemical annotations are partitioned into signed functional polarities²:

- Excitatory connections (+1) encompass acetylcholine (ACh), which dominates sensory and central excitatory transmission, alongside glutamate (Glu, in central circuits), aspartate (Asp), and histamine (His, in primary photoreceptors)¹⁰.
- Inhibitory connections (-1) encompass γ -aminobutyric acid (GABA) and glycine (Gly), which mediate widespread lateral and feedforward inhibition¹⁰.

The signed synaptic weight W_{vu} governing the influence of presynaptic neuron u on postsynaptic neuron v is formulated as the product of the total verified synaptic contact count N_{vu} and the presynaptic polarity sign $S_u \in \{+1, -1\}$, scaled by an empirical unit conductance parameter w_0 ¹⁰:

$$W_{vu} = S_u \cdot N_{vu} \cdot w_0$$

This polarity mapping preserves biologically grounded signal propagation, allowing excitation and inhibition to balance naturally across complex loops without requiring artificial parameter retuning¹⁰.

Whole-Brain Leaky Integrate-and-Fire Dynamics

In work led by Philip Shiu and colleagues, an entire functional brain model of *Drosophila* was realized using a Leaky Integrate-and-Fire (LIF) network derived from the FlyWire connectome².

In this framework, the subthreshold membrane potential $V_i(t)$ of neuron i evolves according to a first-order linear differential equation:

$$\tau_m \frac{dV_i(t)}{dt} = -(V_i(t) - V_{\text{rest}}) + \sum_j W_{ij} s_j(t) + I_{\text{ext},i}(t)$$

where $\tau_m = R_m C_m$ represents the membrane time constant, V_{rest} denotes the resting membrane potential, $I_{\text{ext},i}(t)$ represents sensory or injected currents, and $s_j(t) = \sum_k \delta(t - t_j^k)$ is the presynaptic spike train emitted at times t_j^k . Upon reaching an empirical threshold V_{th} , the neuron fires a discrete spike, drops to a reset potential V_{reset} , and enters an absolute refractory period τ_{ref} .

Shiu et al. demonstrated that this parameterization accurately replicates biological sensorimotor transformations *in vivo* without fitting connection weights to specific behaviors². When simulated current was delivered to gustatory receptor neurons responsive to sweet or water tastants, the excitation cascaded through local interneurons down to proboscis motor neurons MN9 and MN11, which control rostrum lifting and proboscis extension in feeding initiation². Similarly, computational activation of antennal mechanosensory neurons predicted the selective recruitment of the interneuronal grooming cascade, demonstrating that structural connectivity alone largely dictates biological circuit routing².

Beyond integrate-and-fire models, researchers employ connectome-constrained Artificial Neural Networks (ANNs) and linear dynamical systems known as Effectomes²⁰. In these architectures, the connectome acts as a fixed, sparse recurrent adjacency mask¹³. Rather than updating a fully connected layer via backpropagation, deep reinforcement learning updates parameters strictly within biologically verified connections¹⁰. In the Fly-connectomic Graph Model (FlyGM), neurons are partitioned into afferent (sensory), intrinsic (interneuronal), and efferent (motor) sets, demonstrating that models constrained by authentic insect wiring converge faster and achieve lower kinematic tracking error than degree-preserving rewired or random graphs¹⁰.

Embodied Tasks, Gaming Engines, and Closed-Loop Vehicle Operation

Following the public release of the MaleCNS and FlyWire connectomes, software engineers and neuroscientists deployed whole-brain graphs into diverse interactive tasks⁵. Rather than asserting that these simulations recreate biological consciousness, researchers treat the connectome as an evolved, highly efficient computational architecture that can be embedded in closed-loop physical and visual tasks¹.

The sensorimotor cycle begins when external environments—such as the CARLA driving simulator, a VIZDoom combat arena, or the Chromium Dino platform—render raw visual frames or obstacle distances⁷. These environmental signals pass into an engineered sensory encoding layer, where luminance and chromatic features are downsampled and distributed across thousands of outer and inner photoreceptor proxies⁷.

From there, signals propagate through the recurrent, sign-polarized connectome graph, which integrates sensory flow with modulatory inputs like dopamine⁷. Finally, the resulting activation in specialized descending interneurons or motor neurons is extracted by an efferent readout

layer⁷. This readout either directly triggers in-game actions or actuates biomechanical limbs that physically manipulate environmental controls, feeding updated sensory frames back into the network²³.



Driving Autonomous Vehicles in CARLA: The Flyhard Experiment

The Flyhard project, created by Mark Unthank, provides an example of end-to-end embodied mechanics²³. Flyhard interfaces a neural model retaining 165,122 neurons and 25,563,197 synaptic connections from MaleCNS v1.0 with a simulated fly body placed inside the cockpit of a vehicle in the open-source autonomous driving platform CARLA (v0.9.16)²³.

Rather than translating neural firing rates directly into software-level steering variables, Flyhard implements physical causality²³. Camera feeds rendered by CARLA are projected into the fly's visual neuropils, driving neural propagation that reaches motor outputs controlling the fly's thoracic and limb joints²³. The fly's foreleg tarsal segments make physical, friction-based contact with an articulated miniature steering wheel and foot pedals within the virtual cockpit²³.

The angular displacement of this physical wheel dictates the vehicle's turning angle, binding the neural, mechanical, and vehicular dynamics to a single clock²³. In bounded benchmark evaluations on an NVIDIA RTX A6000, a trained connectomic model learned to track target steering angles across 100 out of 100 held-out evaluation targets within 600 optimizer steps, achieving a worst-case hold error of 4.85° (against a threshold limit of 7.45°)²³. Mechanical intervention experiments confirmed that disconnecting the tarsal grip eliminated over 99% of steering responsiveness, demonstrating that vehicle guidance was driven by physical limb interaction rather than unmediated software commands²³.

Interactive Video Games: DOOMFLY, Beat Saber, Fly64, and Fly Dino

Interactive gaming frameworks have served as testing grounds for the sensorimotor capabilities and limitations of biological connectomes:

In the DOOMFLY project developed by Alex Wormuth, the full MaleCNS v1.0 graph operates within the ViZDoom arena⁷. Each rendered frame is downsampled and mapped across 3,335 outer photoreceptor proxies (R1–R6, encoding luminance) and 811 inner photoreceptor proxies (R8, encoding chromatic information)⁷. Firing within visual circuits cascades into descending interneurons, where turning is decoded from the differential activity of **DNp20** (Right minus Left) and forward translation and firing are controlled by **DNpe017**⁷.

To introduce operant conditioning, in-game damage triggers an artificial current pulse to two **PPL101 dopaminergic neurons**⁷. This dopamine signal drives an adapted plasticity rule across 4,184 synapses between Kenyon cells and Mushroom Body Output Neuron 11 (MBON11)⁷. However, across 3,000 continuous arena rounds, the simulation exhibited zero measurable improvement in strategic survival²⁶. This negative result revealed that while static biological wiring generates immediate exploratory reflexes, higher-order tactical behaviors require broader plasticity rules, temporal credit assignment, and bodily affordances that go beyond simple reward-gated mushroom body depression¹⁸.

In rhythm and action environments, researchers have applied reinforcement learning to connectome graphs to evaluate temporal motor coordination¹⁸. Developer Lyra Bubbles trained a connectomic model on replay buffers from *Beat Saber*, using temporal music markers and directional spatial cues to teach descending motor pathways to hit incoming blocks with rhythmic precision²⁵.

In the Fly64 project, Jessica Paquette connected the MaleCNS graph to the *Super Mario 64* engine, mapping collision feedback and visual obstacles to motor readouts that produced continuous exploratory wall-jumping maneuvers⁵.

Taking a more modular approach, the Fly Dino project by Mert Cobanov extracted an

80-neuron MaleCNS sub-circuit operating under leaky $\tanh$ dynamics¹³. By optimizing a lightweight 243-parameter readout layer via the Cross-Entropy Method (CEM), this model successfully cleared incoming cacti and pterodactyls in the *Chromium Dino* runner¹³. This demonstrated that isolated, anatomically preserved insect microcircuits can perform precise, time-critical sensorimotor tasks without requiring whole-brain simulation overhead¹³.

Simulation Project	Connectome Source	Sensory Ingestion Architecture	Motor Readout Architecture	Training / Plasticity Mechanism	Behavioral Performance & Outcome
Flyhard (Mark Unthank) ²³	MaleCNS v1.0 (165, 122 neurons retained) ²³	Visual scenes rendered from CARLA offscreen cameras ²³	Foreleg tarsal joints physically contacting steering wheel/pedals ²³	Supervised trajectory optimization (600 updates) ²³	100% success on target steering hold ($\leq$ error); physical driving in CARLA ²³
DOOMFLY (Alex Wormuth) ⁷	MaleCNS v1.0 (166, 700 neurons complete) ⁷	3,335 R1–R6 (luminance) + 811 R8 (color) photoreceptors ⁷	Turning: DNp20 (R - L); Movement/ Firing: DNpe017 ⁷	Operant dopamine conditioning: PPL101 pulse to KC-MBON1 1 synapses ⁷	Reflexive combat navigation; zero emergence of long-term tactical survivability ²⁶
Beat Saber (Lyra Bubbles) ²⁵	MaleCNS / FlyWire graph extract ²⁷	In-game note vectors, timing markers, and track position ²⁷	Directional saber swing angles and strike timings ²⁷	Reinforcement learning on replay buffers ²⁷	Coordinated timing and positioning to strike rhythmic targets ²⁷
Fly Dino (Mert Cobanov) ¹³	MaleCNS (80 -neuron functional sub-graph) ¹³	Pixel obstacle distance, width, and scroll velocity ²⁴	Binary jump and duck state decoders ²⁴	Cross-Entropy Method (CEM) on 243-parameter readout ¹³	Reliable continuous obstacle dodging; reproducible

					in-browser benchmarks ²⁴
--	--	--	--	--	--

Optical Ground Truth and Silicon Pathway Visualization

Evaluating computational simulations requires comparing their activity against *in vivo* neurophysiology². Functional optical imaging in behaving animals provides the biological ground truth used to validate dynamic models¹⁶.

Biological Ground Truth: Two-Photon Calcium Imaging

In laboratory settings, neural activity is observed in head-fixed *Drosophila* walking atop an air-suspended, frictionless spherical treadmill (polyurethane ball)²⁹. The ball is tracked by high-speed optical sensors that calculate rotational velocity and update a surrounding panoramic LED or digital projection virtual reality arena in closed loop²⁹. Simultaneously, a two-photon laser-scanning microscope records optical signals through a cuticular window in the head capsule²⁹. Genetically encoded calcium indicators, such as GCaMP6f or GCaMP8m, fluoresce in response to calcium influx during action potentials²¹. A prominent biological benchmark is the fruit fly's central compass, located in the central complex²⁹. Here, **E-PG (Ellipsoid body–Protocerebral bridge–Gall)** neurons arborize throughout the toroidal ellipsoid body²⁹. When the fly is imaged on the air-suspended ball, two-photon microscopy reveals a single localized "bump" of calcium activity that travels around the ring of E-PG neurons²⁹. This bump functions as a ring attractor that tracks the animal's azimuthal heading relative to visual landmarks²⁹. In total darkness, the bump continues to track the fly's heading by integrating self-motion (proprioceptive) cues via angular velocity inputs, providing a clear biological target for *in silico* validation⁹.



As the fly rotates on the ball, the activity bump shifts along the ring to maintain heading

In Silico Telemetry and 3D Dynamic Pathway Visualization

In silico simulations offer an advantage over biological imaging: they provide access to every neuron's membrane potential without optical scattering, depth penetration limits, or motion artifacts²⁰. However, rendering 166,700 dynamic neurons at 60 frames per second introduces considerable computational bottlenecks⁵. Modern pipelines rely on specialized visualization workflows:

To resolve network-level dynamics without anatomical clutter, researchers apply Principal Component Analysis (PCA) along feature dimensions to compress multi-channel membrane states ($\mathcal{T}$ time steps $\times$ $\mathcal{N}$ neurons $\times$ $\mathcal{C}$ channels) into a single-channel

representation intensity²⁰. Normalizing this value to a $[0, 1]$ interval yields a scalar proxy for neural activity that can be plotted across force-directed graph layouts²⁰. In these layouts, synaptic connection density—rather than physical anatomy—determines spatial clustering, clearly separating sensory, intrinsic, and descending motor flows¹⁰.

For anatomical 3D rendering, browser-based frameworks use WebGL and Three.js through platforms like the fly-connectome-template¹³. The 3D Cartesian coordinates of cell bodies (somas) and neuropil surface meshes are imported into GPU memory as instanced point-cloud buffers¹³. Voltage or spike vectors from the simulation engine are transmitted across local

WebSockets as compact 8-bit unsigned integer arrays ($0 \rightarrow 255$)²⁰.

A custom vertex and fragment shader pairs these values with an activity color map: resting

neurons remain dark purple ($V_{\text{rest}} \approx -70 \text{ mV}$), transitioning to cyan under subthreshold depolarization, and flashing bright yellow-gold upon reaching the action potential threshold ($V_{\text{th}} \approx -50 \text{ mV}$)³. This pipeline allows observers to watch functional sensory cascades

traverse the brain in real time as the virtual fly navigates its environment¹³.

Practical Implementation Guide: Constructing a *Drosophila* Simulation

Building a custom *Drosophila* simulation involves establishing a software environment, extracting connectomic sub-circuits, running biophysical dynamics, and closing the sensorimotor loop in a physics simulator¹².

Computing Environment and Dependencies

A standard development environment requires Python 3.10+, Node.js 20+ (for browser-based WebGL interfaces), and the MuJoCo 3.x physics engine¹³. Key software packages include flygym for embodied modeling, neuprint-python and fafbseg for connectome querying, and torch for vectorized dynamic updates¹².

Bash

```
# Initialize isolated conda environment
conda create -n flysim python=3.10 -y
conda activate flysim

# Install physics engine and neuromechanical platform
pip install mujoco flygym gymnasium

# Install connectome query tools and tensor backend
pip install navis fafbseg neuprint-python torch torchvision
```

Querying and Filtering Connectomic Subgraphs

Simulating all 166,700 neurons simultaneously requires high-end GPU clusters⁵. For local development, extracting verified sub-graphs—such as the central complex compass or the antennal olfactory system—provides an efficient entry point¹³. The following script demonstrates querying the public Janelia NeuPrint API to build an adjacency matrix with signed synaptic weights:

Python

```
import numpy as np
from neuprint import Client, fetch_adjacencies, NeuronCriteria as NC

# Initialize NeuPrint client for public Janelia connectome dataset
client = Client('neuprint.janelia.org' dataset='hemibrain:v1.2.1' token='YOUR_API_TOKEN')

# Query E-PG compass neurons within the Central Complex
criteria = NC(type='EPG.*')
neuron_df = client.fetch_neurons(criteria)
neuron_ids = neuron_df['bodyId'].tolist()
id_map = {body_id: idx for idx, body_id in enumerate(neuron_ids)}

# Fetch directed synaptic connections between queried neurons
conn_df = fetch_adjacencies(NC(bodyId=neuron_ids), NC(bodyId=neuron_ids))

# Construct signed weight matrix based on neurotransmitter polarities
weight_matrix = np.zeros((len(neuron_ids), len(neuron_ids)), dtype=np.float32)
```

```

polarity_map = {'acetylcholine': 1.0, 'glutamate': 1.0, 'gaba': -1.0, 'glycine': -1.0}

for _, row in conn_df.iterrows():
    pre_id, post_id, count = row['bodyId_pre'], row['bodyId_post'], row['weight']
    if pre_id in id_map and post_id in id_map:
        nt = neuron_df.loc[neuron_df['bodyId'] == pre_id]['predictedNt'].values[0] if 'predictedNt'
in neuron_df else 'acetylcholine'
        sign = polarity_map.get(str(nt), lower()) * 1.0
        weight_matrix[id_map[post_id], id_map[pre_id]] = count * sign

```

Vectorized Spiking Dynamics Engine

Vectorizing the leaky integrate-and-fire equations in PyTorch allows dynamic simulations to execute efficiently on standard consumer GPUs or Apple Silicon hardware¹³:

```

Python
import torch
import torch.nn as nn

class ConnectomeLIFEngine(nn.Module):
    def __init__(self, weight_matrix: np.ndarray, dt=0.001, tau_m=0.02, v_rest=-0.070, v_th=-0.050,
v_reset=-0.075):
        super().__init__()
        self.dt = dt
        self.tau_m = tau_m
        self.v_rest = v_rest
        self.v_th = v_th
        self.v_reset = v_reset
        self.num_neurons = weight_matrix.shape[0]

        # Fixed connectome synaptic adjacency matrix
        self.register_buffer('W', torch.tensor(weight_matrix, dtype=torch.float32))

        # State buffers for membrane potential and binary spike activity
        self.register_buffer('V', torch.full((self.num_neurons,), self.v_rest, dtype=torch.float32))
        self.register_buffer('spikes', torch.zeros((self.num_neurons,), dtype=torch.float32))

    def forward(self, external_current: torch.Tensor):
        # Calculate presynaptic current: W * spikes
        synaptic_input = torch.matmul(self.W, self.spikes)

```

```

        # Euler integration step for membrane potential
        leak = -(self.V - self.v_rest)
        dV = (leak + synaptic_input + external_current) * (self.dt / self.tau_m)
        self.V = self.V + dV

        # Threshold detection and voltage reset
        self.spikes = (self.V >= self.v_th).to(torch.float32)
        self.V = torch.where(self.spikes.bool(), self.v_reset, self.V)

    return self.spikes, self.V

```

Closed-Loop Embodiment in FlyGym

Connecting the spiking dynamics engine to the FlyGym physics environment closes the sensorimotor loop, allowing sensory inputs to drive locomotion in a physical world⁸:

```

Python
from flygym import Fly, SingleFlySimulation
from flygym.preprogrammed import all_leg_dofs

# Initialize the NeuroMechFly digital twin with ommatidial vision
fly = Fly(enable_vision=True, enable_olfaction=False)
sim = SingleFlySimulation(fly=fly)
obs, info = sim.reset()

lif_model = ConnectomeLIFEngine(weight_matrix)

# Run closed-loop simulation loop
for step in range(1000):
    raw_retina = obs['vision'] # Ommatidial luminance arrays

    # Inject sensory current into visual-receptive neurons
    ext_drive = torch.zeros(lif_model.num_neurons)
    ext_drive[0] = float(np.mean(raw_retina[0])) * 0.05
    ext_drive[1] = float(np.mean(raw_retina[1])) * 0.05

    # Update spiking dynamics
    spikes, voltages = lif_model(ext_drive)

```

```

# Map descending motor neuron activity to physical leg joint kinematics
turn_bias = (spikes[1] - spikes[2]) / item()

action = {dof: np.sin(step * 0.1 + 0.2 * turn_bias) for dof in all_leg_dofs}

# Advance physics engine step
obs, reward, terminated, truncated, info = sim.step(action)

if terminated or truncated:
    break

sim.close()

```

Real-Time Shader Pipeline for 3D Activity Rendering

For real-time pathway rendering, the simulation passes voltage vectors to a WebGL client via WebSockets¹³. The frontend uses Three.js instanced meshes positioned at the soma coordinates, updating colors through a fragment shader to display neural activity dynamically:

```

OpenGL Shading Language
precision mediump float;
varying float intensity;

void main()
{
    // Resting state: dark purple; Intermediate: cyan; Spiking state: bright yellow
    vec3 restColor = vec3(0.08, 0.02, 0.18);
    vec3 midColor = vec3(0.00, 0.70, 0.85);
    vec3 spikeColor = vec3(1.00, 0.88, 0.10);

    vec3 finalColor = vec3(intensity * 0.5);
    finalColor = mix(restColor, midColor, intensity * 2.0);
    finalColor = mix(midColor, spikeColor, (intensity * 0.5) * 2.0);

    gl_FragColor = vec4(finalColor, 1.0);
}

```

Synthesis and Future Outlook

Embodied simulations of *Drosophila melanogaster* bridge the gap between static structural anatomy and dynamic behavioral neuroscience¹⁰. By embedding connectome graphs within physical engines like MuJoCo and CARLA, researchers have shown that biological wiring provides a powerful structural prior for sensorimotor control¹⁰. Connectome-constrained

networks consistently learn faster and produce more stable locomotion than unconstrained artificial architectures¹⁰.

However, projects like DOOMFLY demonstrate the clear limitations of static connectomics⁷.

Without slow-timescale neuromodulatory dynamics, neuropeptide diffusion, activity-dependent structural plasticity, and bodily affordances, a static connectome functions as a complex reflexive controller rather than a general-purpose learning system¹⁸.

Advancing the field will require incorporating non-synaptic volume transmission, biophysical muscle dynamics, and unified models that encompass the entire nervous system⁴. As open-source platforms like FlyGym and community connectomic workbenches continue to evolve, *Drosophila* models offer an accessible foundation for reverse-engineering biological intelligence and developing neuromorphic, embodied artificial agents⁸.

Works cited

1. Google's Simulated Fruit Fly AI Learns to Drive, Park & Game | Stork.AI, <https://www.stork.ai/blog/this-ai-fly-drives-better-than-you>
2. A Drosophila computational brain model reveals sensorimotor, <https://www.repository.cam.ac.uk/items/f6d6fd8e-ffa0-4e9f-aaca-71732c523253>
3. Researchers simulate an entire fly brain on a laptop. Is a human, <https://news.berkeley.edu/2024/10/02/researchers-simulate-an-entire-fly-brain-on-a-laptop-is-a-human-brain-next/>
4. The Adult Drosophila Connectome Ecosystem, <https://flyconnecto.me/2026/09/04/the-adult-drosophila-connectome-ecosystem/>
5. Google maps entire brain and central nervous system of adult male, <https://www.tomshardware.com/software/programming/google-maps-entire-brain-and-central-nervous-system-of-adult-male-fruit-fly-software-engineers-immediately-make-it-run-doom-ai-powered-3d-model-of-over-166-000-neurons-can-also-play-super-mario-64>
6. Three Days After Google Mapped a Fly Brain, It Plays Doom - Aixploria, <https://www.aixploria.com/en/ai-radar/fly-brain-connectome-doom-mario-64/>
7. GitHub - nftechie/doomfly: Fly-connectome simulation controlling a, <https://github.com/nftechie/doomfly>
8. Simulating embodied sensorimotor control with NeuroMechFly v2, <https://github.com/NeLy-EPFL/flygym/>
9. NeuroMechFly v2: Simulating how fruit flies see, smell, and navigate, <https://www.eurekalert.org/news-releases/1064414>
10. Whole-Brain Connectomic Graph Model Enables Whole-Body, <https://arxiv.org/html/2602.17997v3>
11. NeuroMechFly, https://neuromechfly.org/api_reference/flygym/compose/fly/neuromechfly/
12. NeuroMechFly documentation - GitHub Pages, <https://nely-epfl.github.io/flygym-gymnasium/>
13. GitHub - cobanov/awesome-fly: A curated list of fruit fly connectome,

- <https://github.com/cobanov/awesome-fly>
14. Compiling a Vector Programming Language to the Drosophila,
<https://clawrxiv.io/abs/2604.01566>
 15. Can a fruit fly run a company better than I can? - Buğra Tezcan,
<https://bugratezcan.com/posts/fruit-fly-ceo/>
 16. (PDF) A Drosophila computational brain model reveals sensorimotor,
https://www.researchgate.net/publication/384573114_A_Drosophila_computational_brain_model_reveals_sensorimotor_processing
 17. NeuroMechFly, <https://neuromechfly.org/>
 18. After Google mapped an adult male fruit fly's brain, software ... - Reddit,
https://www.reddit.com/r/gaming/comments/1wcfks2/after_google_mapped_an_adult_male_fruit_flys/
 19. Prediction of neural activity in connectome-constrained recurrent,
https://www.researchgate.net/publication/396947961_Prediction_of_neural_activity_in_connectome-constrained_recurrent_networks
 20. Whole-Brain Connectomic Graph Model Enables Whole-Body,
<https://arxiv.org/html/2602.17997v1>
 21. Infrequent strong connections constrain connectomic predictions of,
<https://www.biorxiv.org/content/10.1101/2025.03.06.641774.full>
 22. Fruit Fly Brain Uploaded to AI: How the Connectome Project Works,
<https://www.mindstudio.ai/blog/fruit-fly-brain-connectome-ai-training>
 23. GitHub - MarkUnthank/flyhard: A connectome-based fruit fly,
<https://github.com/MarkUnthank/flyhard>
 24. cobanov/flyjump: Fly Dino: original Chromium Dino ... - GitHub,
<https://github.com/cobanov/flyjump>
 25. The Latest Weird Thing to Play Doom Is the Mapped-Out Brain of a,
<https://www.cnet.com/tech/gaming/simulated-fruit-fly-brain-plays-doom/>
 26. DOOM: A 166,700-Neuron Simulated Fruit Fly Brain Has Learned to,
<https://retrogems.fr/en/simulated-fruit-fly-brain-plays-doom/>
 27. <https://www.pcgamer.com/hardware/after-google-mapped-an-adult-male-fruit-flys-brain-software-engineers-made-it-play-doom-mario64-and-beat-saber/>
 28. Awesome list of fruit-fly connectome simulations, demos ... - GitHub,
<https://github.com/townie/awesome-fruit-fly>
 29. Neural dynamics for landmark orientation and angular path integration,
https://www.researchgate.net/publication/276360593_Neural_dynamics_for_landmark_orientation_and_angular_path_integration
 30. Drosophila behavioral paradigms - Society for Developmental Biology,
https://www.sdbonline.org/sites/fly/aimain/6locomotive_behavior.htm
 31. Bitcoin's Weirdest Trader Yet? Fly Brain Makes a Profit - TradingView,
https://www.tradingview.com/news/u_today:6e55cdca1094b:0-bitcoin-s-weirdest-t-trader-yet-fly-brain-makes-a-profit/
 32. 6. Muscle-based imitation learning - NeuroMechFly,
https://neuromechfly.org/tutorials/6_muscle_imitation/